In-kernel	Kernel bypass / userspace
eBPF kernel programs [2] XDP (native/generic) [2] Linux network stack (TCP/IP) [5] AF_PACKET, PACKET_MMAP [1, 3, 4]	DPDK [1, 2, 3, 4] AF_XDP [1, 2, 3, 4] CNDP over AF_XDP [1] NIC-vendor bypass (mlx5 Direct Verbs, verbs, ef_vi) [3, 4] RoCEv2 RDMA, NIC-offloaded [5] uXDP over DPDK/AF_XDP [2]

Paper	Approaches	Workload	Metrics	Comparison basis
UPF I/O modes (2026) [1]	AF_PACKET, AF_XDP zero-copy, CNDP, DPDK PMD as BESS ports behind `RecvPackets` and `SendPackets`	5G UPF pipeline (GTP-U, PDR/FAR/QER), 64–1518 B, 1–8 workers, 24-h soak	Non-drop rate (RFC 2544), in-pipeline latency percentiles, per-core rate, worker scaling	Same host, NIC, kernel and UPF image, only the port driver changes. Different generators per mode. AF_XDP/CNDP capped at two workers.
uXDP (2025) [2]	Native/generic XDP, userspace eBPF via DPDK/AF_XDP	Counters, filtering, load balancing, Katran, MAC-swap echo (`xdp_tx`)	NF throughput, unloaded RTT, compiler/inlining effects	Same programs/hardware, one core. Mode comparison: optimised source LLVM IR.
packetio (2026) [3]	AF_PACKET, AF_XDP, DPDK, mlx5 Direct Verbs behind one Go API	Transmit, receive, L3 LPM forward, 64 B at 100 G	Mpps per core, cores to line rate incl. softirq, median RTT (mlx5 only)	Same sweep program, backend by build tag. No XDP, no partial load, no energy.
ABTRDA3 (2026) [4]	ef_vi, DPDK, AF_XDP, verbs, PACKET_MMAP behind C++20 ring concepts	MAC-swap reflector, one frame in flight, 24-h soak	RTT median to P99.999	Same loop, transport by config file. Latency only.
Fast but Wasteful (2026) [5]	Linux TCP/IP stack, RoCEv2 RDMA; core governors and fixed uncore frequency as variables	`iperf3` TCP flows, Valkey key-value store under YCSB, `perftest` RDMA flows, 100 G	Throughput per utilised core, KRPS vs. P50/P99 latency, active wall power (`ipmitool`), core/uncore frequency (PCM)	Same two servers, only governor and uncore frequency change. No packet framework compared, NIC power not reported.

Comparing UPF Dataplane I/O Modes in a Cloud-Native 5G Core: AF_PACKET, AF_XDP, CNDP, and DPDK on SD-Core BESS-UPF [1]

One UPF pipeline, four I/O backends behind the same BESS port primitives: at two workers AF_XDP, CNDP and DPDK are within measurement noise and AF_XDP leads per core, DPDK's lead is a scaling-ceiling effect, and AF_PACKET is an order of magnitude slower.

Earlier work compares I/O modes in pairs, most often AF_XDP against DPDK, or with synthetic generators, so UPF operators have to combine results from different testbeds and applications before they can decide. Here one open-source 5G user plane function (SD-Core BESS-UPF) runs under four I/O modes and only the port from which packets are received changes, so the I/O backend is the only variable. Measured are throughput, latency, CPU use and stability; deployment requirements, operational pitfalls and a decision guide are reported next to the measurements (Sections 3, 7 and 9).

Port abstraction

- *BESS port as black box.* The Berkeley Extensible Software Switch hides the port from the userspace pipeline: the UPF only calls `RecvPackets` and `SendPackets` on packet batches. A configuration file selects driver (for example the DPDK poll-mode driver), ring and kernel involvement; all of it stays invisible to the UPF code.
- *Four modes.* AF_PACKET and AF_XDP through DPDK poll-mode drivers, CNDP through its own port type, DPDK through a native PMD over `vfio-pci`.
- *CNDP.* A userspace abstraction on top of AF_XDP with a DPDK-like programming model (mempools, ports) but without DPDK's hugepage and VFIO requirements.

Metrics

- *Throughput.* Non-drop rate per RFC 2544 over 64–1518 B (Figure 18, Tables 3 and 4). At 64 B: CNDP 5.52 and AF_XDP 6.47 Mpps with two workers, DPDK 10.30 Mpps with four and 13.09 Mpps with eight; AF_PACKET 0.25 Mpps at 86 B.
- *Latency and jitter.* In-pipeline probes give mean, P50, P99 and P99.9 (Table 5): DPDK 8.1 µs mean, CNDP and AF_XDP 13–14 µs. A second AF_XDP worker adds P99 jitter through contention on the kernel redirect path (Table 6).
- *CPU and core scaling.* Per-core rate and scaling efficiency (Table 7): AF_XDP 3.69, DPDK 2.99, CNDP 2.65 Mpps per core. A per-netdev AF_XDP socket limit pins CNDP and AF_XDP at two workers.

Workload

- *UPF pipeline.* GTP-U decapsulation, PDR lookup, QER metering and FAR action with a single rule set.
- *Synthetic and real traffic.* `testpmd` and kernel `pktgen` for I/O ceilings; a 24-hour soak and a real-RAN run validate CNDP only.

Comparison basis

Same host, NIC (Intel XXV710), real-time kernel, Kubernetes cluster and UPF image; the port driver is the only variable (Table 2). Caveats: a different load generator per mode, traffic hairpinned inside the NIC because of 1-GbE optics, a NIC receive path that caps every mode near 3 Mpps per core, no energy measurement, no in-kernel XDP path. BESS adds its own scheduler and module pipeline above the port, so the measured path is BESS plus port, not the port alone.

---

uXDP: Frictionless XDP Deployments in Userspace [2]

DPDK-mode uXDP is fastest, up to 3.3× in-kernel XDP, but the figures mix I/O-path and compiler effects.

uXDP runs unchanged XDP/eBPF programs in userspace via DPDK or AF_XDP. It extends `bpftime`, retaining eBPF verification and existing control-plane compatibility. Optimisations: helper/map inlining and SIMD instructions. DPDK mode avoids interrupt-based retrieval and enables execution where kernel eBPF is unavailable.

Deployment modes

- *DPDK mode.* Userspace RX/TX rings and packet processing bypass the kernel network stack. Kernel-bound traffic requires reinjection.
- *AF_XDP mode.* Kernel XDP redirects packets to an AF_XDP socket for userspace eBPF processing. Shared RX/TX rings and packet buffers avoid per-packet copies in zero-copy mode.

In AF_XDP mode, kernel XDP can pass selected packets directly to the Linux stack before userspace processing. After uXDP processing, the evaluated version supports only `XDP_DROP` and `XDP_TX`. Kernel reinjection for `XDP_PASS` is future work (Section 7).

Metrics

- *Throughput across deployment modes.* Modes in Figure 4: DPDK (`dpdk_llvm`), AF_XDP (`afxdp_llvm`), native XDP (`drv_mode`), generic XDP (`skb_mode`). Up to 3.3× throughput over in-kernel execution and up to 40% higher throughput for Meta's Katran. DPDK mode is fastest across tested functions. AF_XDP mode can exceed native XDP for complex functions. Generic XDP is slowest.
- *Unloaded round-trip latency.* `xdp_tx` MAC-swap RTT (Figure 5): DPDK mode has the lowest latency, AF_XDP mode slightly higher latency than native XDP.

Workload

- *XDP network functions.* Packet counters, packet-length statistics, filtering, simple load balancing and Katran, with different helper/map combinations.
- *MAC swap and transmit.* `xdp_tx`: swap source/destination MAC addresses and transmit back (latency benchmark).

Comparison basis

Same programs and testbed, one CPU core. 64-byte TCP or 128-byte ICMP packets. Existing eBPF binaries run without recompilation. Figure 4 uses optimised source LLVM IR: results combine deployment-mode and compiler effects.

Userspace I/O abstractions: packetio and ABTRDA3 [3, 4]

Both expose one API over several ways to reach the NIC, so a generic userspace I/O interface is state of practice; the frame idea goes further by including the in-kernel path, which leaves the overlap large for DPDK and AF_XDP and nil for XDP and Snabb.

Two open-source projects from 2026 without peer review. *packetio* (Go) hides as much setup complexity as possible behind one API that exposes AF_PACKET, AF_XDP, DPDK and mlx5 Direct Verbs; the same sweep program (transmit, receive, L3 forward) runs on all four backends. *ABTRDA3* (C++20) follows the same concept because every kernel-bypass framework has a different setup ritual: the transport is a template parameter behind `TxRing`/`RxRing` concepts, chosen in a TOML file, over ef_vi, DPDK, AF_XDP, verbs and PACKET_MMAP.

Metrics

- *packetio.* Mpps at one core and cores to line rate on an AMD EPYC 9275F with a ConnectX-6 Dx at 100 G, 64-byte frames, measured September 2026: forwarding per core mlx5 29.3, DPDK 22.6, AF_XDP 17.5 (two cores); AF_PACKET collapses under load. Cores are counted machine-wide including softirq work. Latency only as medians on mlx5.
- *ABTRDA3.* 24-hour loopback RTT percentiles up to P99.999 on X2522, ConnectX-4 Lx and XXV710 (ef_vi 1.87, verbs 2.43, DPDK 3.51 µs median). No throughput, CPU or energy.

Comparison basis

Same program per project, backend swapped by build tag or configuration file. No in-kernel XDP, no Snabb, no empty-frame calibration, no partial-load sweep, no energy. Design references for the frame interface (opaque packet handle, backend by configuration, CPU including kernel work), not measurement studies.

Fast but Wasteful: Power Analysis and Implications of Host Networking [5]

Network-intensive load drives the CPU uncore to near-maximum frequency and the uncore never idles, so capping it at 1.6 GHz saves 43–46 % power for 11–12 % less key-value throughput, while `schedutil` and `conservative` under-clock IRQ cores.

Measurement study of CPU frequency scaling, core and uncore, on host networking performance and power for the Linux TCP/IP stack and RoCEv2 RDMA. More than one core is needed to saturate a 100-Gbps link, so DVFS decisions on these cores shape the energy footprint. The uncore (last-level cache, memory controllers, interconnect) is scaled by hardware and firmware with undisclosed logic; the study fixes it by setting equal lower and upper bounds. No packet-processing framework is compared: no XDP, DPDK or AF_XDP.

Power measurement method

- *Wall power.* `ipmitool` reads system power once per second. Reported is active power: wall power under load minus idle power at the lowest core and uncore frequency without workload, which removes temperature and other environment effects.
- *Breakdown.* Intel PCM logs core utilisation, core and uncore frequency, and CPU and DRAM power from RAPL, which excludes fans and NIC. Uncore power cannot be read directly and is projected from an idle sweep over uncore frequency (Figure 1).
- *NIC power.* Not reported: the ConnectX-6 Dx exposes no power counter, its active power is estimated at under 8 W.

Metrics

- *Uncore frequency and power.* Twelve pinned `iperf3` TCP flows push the NIC-local uncore to 2.49 of 2.5 GHz; even the unused NIC-remote socket rises above 2 GHz (Table 2). Uncore accounts for 74 % of sender and 61 % of receiver active power (Figure 3). Fixed at 1.6 GHz, throughput per core drops 2–7 % and power 45–55 %; at 0.8 GHz 13–27 % versus 67–82 % (Figure 2). CPU- and memory-intensive `stress-ng` loads leave the uncore at 0.8–1.3 GHz; only networking drives it to the maximum, plausibly through Direct Cache Access (Figure 4).
- *Core governors.* `ondemand` delivers 61 % more throughput per core than `schedutil` at default uncore for equal power (Figure 2). Boosting IRQ cores to 1.6 GHz under `schedutil` gains 24 % throughput per core for under 2.7 W (Figure 5).
- *Key-value store.* Valkey with 256 B and 4 KB records, 99 % reads: uncore at 1.6 GHz saves 43.3–46.1 % power for 10.9–11.6 % maximum throughput (Figures 6 and 8). At low load, 0.8 GHz matches P50 and P99 latency up to 100 KRPS with 77–84 % less power; the uncore capacity needed for a latency target depends on load, which the default setting ignores.
- *RDMA.* Requester cores busy-poll at full utilisation and maximum frequency, so governors have no effect. Responder power follows the uncore, which bandwidth-intensive flows raise above about 51–53 Gbps while message-intensive flows at 55 Mmps do not (Figures 10 and 11). Uncore at 0.8 GHz adds at most 0.6 µs message latency (Figure 9); RDMA-based Valkey loses 23.8 % throughput for 72 % less power (Table 3).

Workload

- *TCP.* `iperf3`, one flow and one IRQ thread per core, twelve flows on 24 of 28 cores, 60 s, NIC-local socket only. The link is the bottleneck, so aggregate throughput divided by total core utilisation is the metric.
- *Key-value store.* Valkey server under YCSB load sweeps, Zipfian and uniform keys, 24 client processes with ten threads each; RDMA variant with 1 MB SET values.
- *RDMA.* `perftest` WRITE and READ, bandwidth-intensive (1 MB messages) and message-intensive (32-batch of 16 B), 1–24 flows, one core per flow.

Comparison basis

Two identical servers, 2× Xeon Gold 5520+ (core 0.8–2.0 GHz, uncore 0.8–2.5 GHz), ConnectX-6 Dx at 100 G back to back, Ubuntu 24.04, Turbo Boost and Hyperthreading off (Table 1). Variables are the governor (`schedutil`, `ondemand`, `conservative`) and the fixed uncore frequency; workload and stack stay constant. For the thesis: template for the energy measurement (wall power minus idle, 1-s sampling, RAPL for the breakdown) and the two tuning profiles, since an untouched system sets uncore frequency and governor differently per load.

References

[1] S. Valia, N. Rajput, A. Gairola and V. Rathi. *Comparing UPF Dataplane I/O Modes in a Cloud-Native 5G Core: AF_PACKET, AF_XDP, CNDP, and DPDK on SD-Core BESS-UPF*. arXiv:2608.14589, 2026. arXiv.

[2] Y. Zheng, P. Gavriil and M. Kogias. *uXDP: Frictionless XDP Deployments in Userspace*. eBPF Workshop, 2025. DOI.

[3] A. Toonk. *packetio: one API over four ways to reach the NIC*. GitHub repository and blog, 2026. Code, Blog.

[4] A. Sherjil. *ABTRDA3: C++20 kernel-bypass networking library with 24-hour tail-latency benchmarks*. GitHub repository, 2026. Code.

[5] J. Ok and S. Moon. *Fast but Wasteful: Power Analysis and Implications of Host Networking*. Proceedings of the ACM on Networking (CoNEXT), 2026. DOI.